同程面试

架构设计　面试题　动态表单　数据迁移　动态服务

- 同程面试
  - 1. 动态表单初稿
    - 1.1 背景
    - 1.2 需求
    - 1.3 方案选择
    - 1.4 纵向属性设计
      - 1.4.1 架构图
      - 1.4.2 表结构
    - 1.5 数据迁移设计
      - 1.5.1 迁移接口
      - 1.5.2 影响范围
      - 1.5.3 执行要点
      - 1.5.4 回滚策略
      - 1.5.5 数据验证
  - 2. 动态服务初稿
    - 2.1 背景
    - 2.2 需求
    - 2.3 方案选择
      - 2.3.1 配置中心
  - 3. 附录一、属性类型

1. 动态表单初稿

1.1 背景

随着系统日益壮大，客户的需求越来越个性化，面对大量的工单需求，现有系统显得力不从心。目前的工单结构单一，只能通过硬编码的实现新需求，这将消耗大量的人力物力，影响了项目的正常迭代。总结痛点：

- **开发周期长**：有新的表单需求时，开发要排期，用户要等待，大量的表单开发会影响项目迭代计划；
- **系统越来越重**：频繁的表字段变更、新增，使得系统越来越重。
- **周期性发版**：开发完成需要发版，存在不稳定因素，影响用户体验。

综上，为解决上述问题，本小节试图设计一个工单维护模块，来统一管理工单模型。

1.2 需求

1. 可动态装配个性化表单，支持大部分表单业务；
2. 支持快速、准确的查询和统计工单；
3. 不影响历史数据，旧数据做迁移；

1.3 方案选择

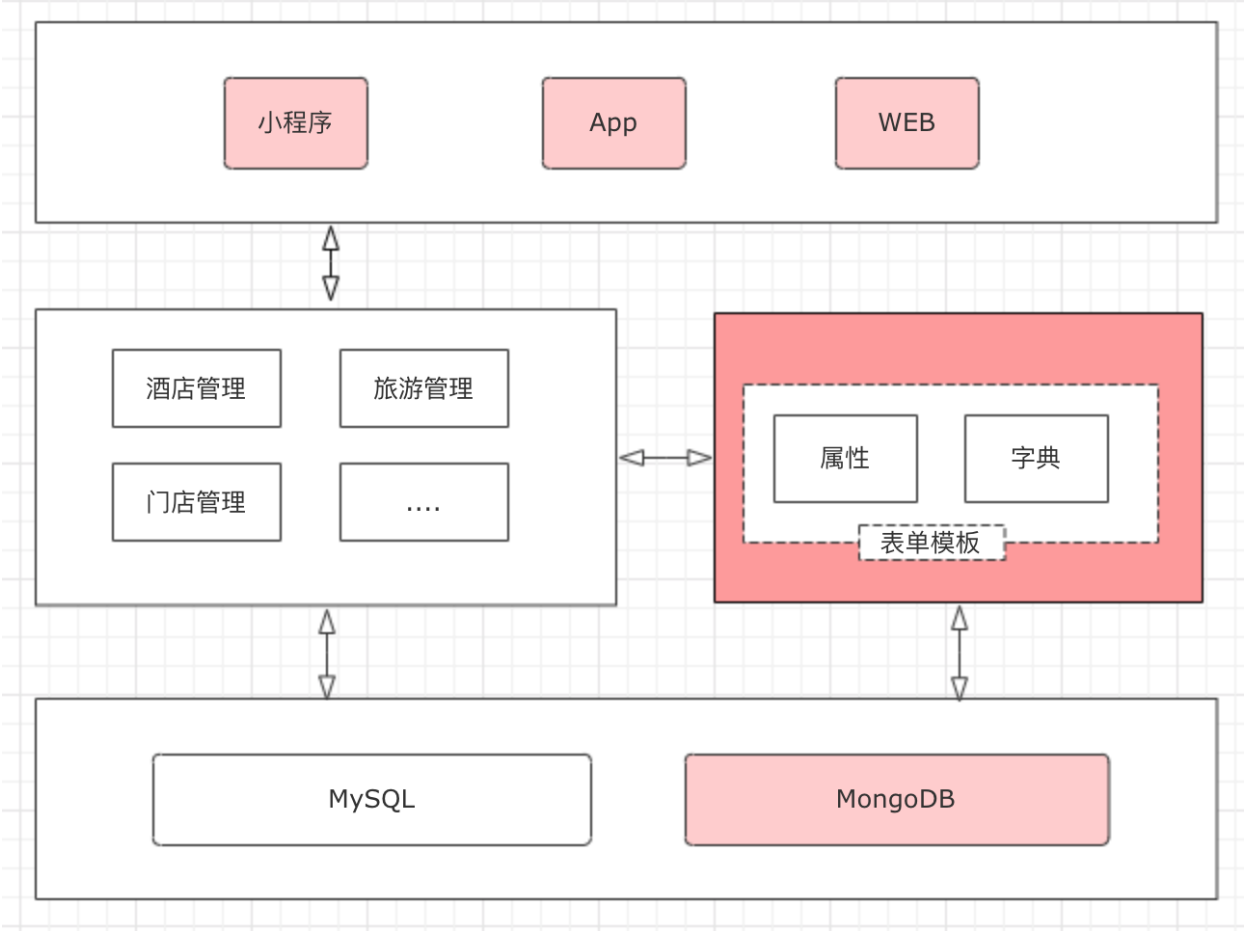
是否启用	方案	优点	缺点
☐	动态添加新字段	开发难度低	需要开发，工期长
☐	设计时预留字段	开发难度低	字段枚举不全
☐	使用JSON保存数据	开发难度低，一个字段搞定所有	字段大、数据冗余
☑	纵向属性	可扩展、可维护、零编码	开发复杂、元数据模型决定系统质量

综上，第四种方案【纵向属性】比较适合当前需求。

1.4 纵向属性设计

动态表单的核心是可接受任意风格的数据录入，那么，设计的核心就是表单建模的功能设计。

1.4.1 架构图



1.4.2 表结构

```
-- 属性表
+──────────────────+
| property         |
+──────────────────+
| id               |
| name             |
| label            |
| description      |
| type             |
| unit             |
| regex            |
+──────────────────+

-- 属性字典表
+────────────────────+
| property_meta      |
+────────────────────+
| id                 |
| property_id        |
| value              |
+────────────────────+

-- 表单模板表
+──────────────────+
| template         |
```

```
+──────────────────+
| id               |
| name             |
| description      |
| creator          |
+──────────────────+

-- 表单属性关系表
+──────────────────────+
| template_property    |
+──────────────────────+
| id                   |
| property_id          |
| order                |
| label                |
| default              |
| required             |
| location             |
| event                |
| .....                |
+──────────────────────+
```

以上列出了表单模板的表结构，对于表单的明细数据可存放在MongoDB中，方便查询和统计。

1.5 数据迁移设计

为了降低迁移的风险，保护数据的完整性，这次迁移需支持增量，如果数据已经迁移过，那么这条数据跳过迁移逻辑。

> TODO：补充迁移关系图

- [ ] 横向数据 –> 属性表
- [ ] 横向数据 –> 属性字典表
- [ ] 横向数据 –> 表单模板表（可人工干预）
- [ ] 横向数据 –> 表单属性关系表
- [ ] 横向数据 –> 明细表

1.5.1 迁移接口

系统开发完成后，为每个酒店集团，或者每种表单风格建立好表单模板，迁移时指定模板Id以及旧的表单风格唯一标识，

按酒店迁移，提供一个迁移接口即可，接口需指定表单模板和酒店id入参。

> TODO：接口设计

1.5.2 影响范围

如果迁移中出现问题，原有数据不会被删除，新数据未正式向用户开放，可以手动处理，无其他影响。

1.5.3 执行要点

执行建表语句> 后端发版> 执行迁移> 数据验证> 前端发版> 完成迁移

1.5.4 回滚策略

旧表的数据不删除，新表数据增量迁移，无需回滚。

1.5.5 数据验证

迁移完毕后，测试验证，如果没有问题，注释迁移接口即可。

2. 动态服务初稿

2.1 背景

紧张的项目排期使得模块零散、混杂，伴随着业务的逐渐增多，这个问题日益突出。服务越来越多，系统越来越大。遵循单一职能、统一管理原则才会使项目更加清晰，减少开发中浪费不必要的时间。

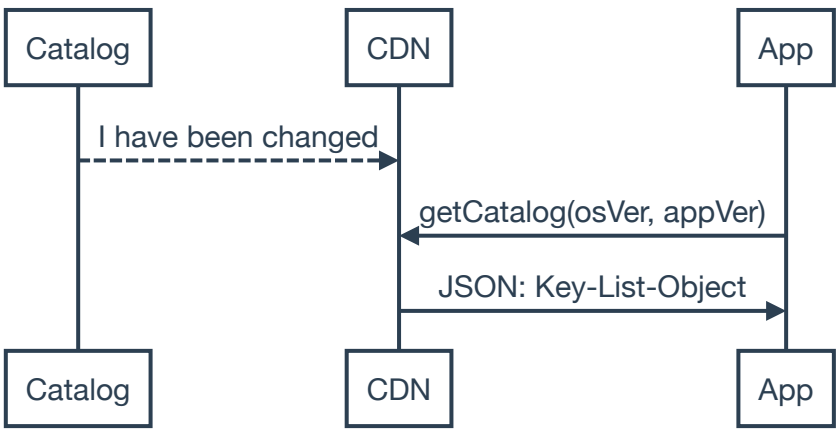
2.2 需求

1. 可统一管理现有模块；
2. 可动态添加新业务模块；

2.3 方案选择

2.3.1 配置中心

为了实现上述需求，站在模块更上层来考虑这个问题，本小节试图设计一个服务，来统一应用的配置和维护，项目取名：Catalog



工作流程:

1. 在Catalog建立配置项，配置项发生改变时推送到CDN；
2. 应用启动时从CDN获取配置文件；
3. 根据配置文件动态渲染页面；

这个服务可以是公司级的项目目录，支持按项目、按平台、按版本、按模块来配置更细粒度的数据甚至是UI界面。
新发布一个模块，在这里配置数据来源，依赖的接口服务，这个设计不关心模块怎么部署，只要模块能对外暴露数据即可。

增量更新:
随着业务的增长，配置中心的数据尺寸愈发膨胀，每次启动应用都将拉取完整的配置文件，严重影响应用体验。
当数据请求到达配置中心后，系统将比对当前最新版本和请求的osVer（系统版本）、appVer（应用版本），并计算出版本差之间发生变更的模块数据，构造一次增量数据返回。

3. 附录一、属性类型

类型	控件名
input	文本框
select	选择框
textarea	文本域
date	日期
datetime	时间空间
file	文件
checkbox	多选框
radio	单选框